

Escrow e macchina a stati

Progettare le transizioni prima del codice

`Created` · `Funded` · `ReleaseApproved` · `Cancelled`

Contratto giocattolo, volutamente incompleto:
riceve ETH, ma non esegue alcun payout. Non production-ready.

Baseline: Solidity 0.8.37
verificata il 21 settembre 2026

1 • OBIETTIVI

Cosa saprai fare alla fine della lezione

Pensare per stati

Distinguere stato, transizione, guardia, azione e stato terminale, e tradurre una specifica di Escrow in un grafo.

Scrivere le guardie

enum per gli stati discreti, immutable per i parametri fissati alla costruzione, controlli di ruolo e di stato prima di toccare lo storage.

Riconoscere i bug di sequenza

Individuare una transizione consentita nel momento sbagliato e costruire una transition matrix da auditor.

Testare le frecce proibite

Test per il percorso corretto e per i percorsi vietati, verificando che un revert non lasci stato parzialmente modificato.

IL PRINCIPIO GUIDA DELLA LEZIONE

Non chiederti soltanto «questa funzione è corretta?». Chiediti: da quali stati è raggiungibile, da chi, e quali stati rende raggiungibili dopo di sé?

2.1 • MODELLO MENTALE

Non solo «cosa fa», ma «quando è consentito»

UN DISTRIBUTORE AUTOMATICO

```
nessuna moneta inserita → selectProduct() NO  
moneta inserita        → selectProduct() può funzionare  
prodotto erogato       → selectProduct() NO, di nuovo
```

La stessa funzione può essere corretta in uno stato ed errata in un altro, o pericolosa se può essere ripetuta.

LO STORAGE SOPRAVVIVE ALLE TRANSAZIONI

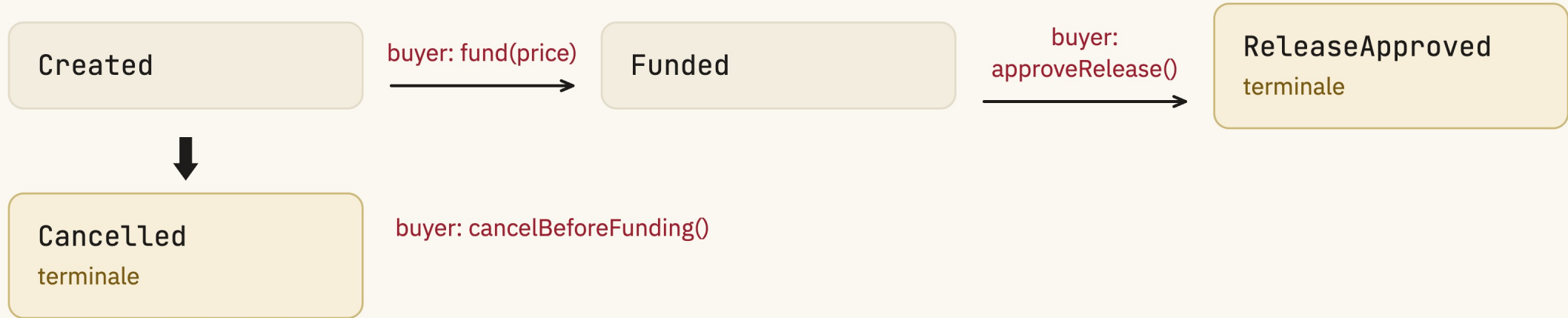
```
Tx 1 → modifica storage  
Tx 2 → parte dallo storage di Tx 1  
Tx 3 → parte dallo storage di Tx 2
```

La sicurezza non dipende solo dal singolo input, ma dalla **sequenza di chiamate**.

OWASP Smart Contract Security classifica le vulnerabilità di business logic come difetti in cui regole, incentivi o transizioni permettono comportamenti non previsti, anche quando type safety e access control sembrano corretti.

2.2 · LA STATE MACHINE DI OGGI

Quattro stati, tre transizioni valide



2.2 · IL NEGATIVE SPACE

Tutto ciò che non compare nel grafo è proibito

Funded → Cancelled

Created →
ReleaseApproved

Cancelled → Funded

ReleaseApproved →
Funded

Funded → Funded

Una specifica di state machine è definita tanto dalle frecce presenti quanto dalle frecce assenti.

Da auditor, ogni freccia mancante è una proprietà da testare: qualcuno prima o poi proverà a percorrerla.

2.3 · STATO, GUARDIA, AZIONE

Ogni transizione ha la stessa forma

```
(current state, caller, inputs, value)
```

|

v

GUARDIE

|

se tutte vere

v

AZIONI

|

v

new state

FUND(), NEL DETTAGLIO

stato attuale = Created

caller = buyer

msg.value = price esatto

↓

state = Funded

SE UNA GUARDIA FALLISCE

revert

La transazione non deve lasciare effetti parziali.

2.4 • DUE DIMENSIONI DIVERSE

«Chi?» e «quando?» non sono la stessa domanda

```
function approveRelease() external {  
    require(msg.sender == buyer);  
    state = State.ReleaseApproved;  
}
```

AUTORIZZAZIONE

Chi può fare l'azione?

Qui è verificata.

SEQUENZIAMENTO

In quale stato può farla?

Qui manca del tutto.

Il buyer è il soggetto giusto. Ma se può chiamare approveRelease() quando l'Escrow è ancora Created, la state machine è comunque sbagliata.

3.1 • FORMALIZZARE

Prima la specifica, poi il codice Solidity

S = insieme degli stati

T = transizioni consentite

G = guardie che le abilitano

A = azioni della transizione

Questa forma separa la **specifica** dall'implementazione: prima decidiamo cosa deve essere vero, poi scriviamo il codice.

ID	Transizione	Guardie principali
T1	Created → Funded	msg.sender == buyer · msg.value == price · state == Created
T2	Funded → ReleaseApproved	msg.sender == buyer · state == Funded
T3	Created → Cancelled	msg.sender == buyer · state == Created

3.2 · BUG DI SEQUENZA

Una riga innocua, una storia impossibile

```
state = State.ReleaseApproved;
```

```
deploy  
↓  
Created  
|  
| approveRelease() ← se manca il controllo di stato  
↓  
ReleaseApproved
```

«Il buyer ha approvato il rilascio di
fondi già messi in escrow»
ma non è mai avvenuto alcun funding.

LA DOMANDA DA AUDITOR

Quali eventi devono necessariamente essere già
avvenuti perché questo stato sia valido?

3.3 • ENUM

Un insieme finito di valori, leggibile in review

```
enum State {  
    Created,  
    Funded,  
    ReleaseApproved,  
    Cancelled  
}  
  
state = State.Funded
```

L'ALTERNATIVA, PIÙ AMBIGUA

```
uint8 state;  
// 0 = Created  
// 1 = Funded  
// 2 = ReleaseApproved
```

Audit e code review sono attività umane: ridurre l'ambiguità aiuta a individuare gli errori.

Un enum **non rende corretta** la state machine: il compilatore non sa che Created → ReleaseApproved è semanticamente sbagliato. Quella regola vive nelle guardie e nei test.

3.4 · STORAGE E IMMUTABLE

Cosa cambia nel tempo e cosa è fissato alla nascita

`State public state;`

State variable ordinaria: il valore vive nello **storage persistente**.

```
Tx A termina con state = Funded  
Tx B inizia e legge ancora Funded
```

È il motivo per cui la macchina a stati esiste tra transazioni diverse.

`address public immutable buyer;`

Anche seller e price. Fissati nel constructor, incorporati nel runtime code invece di occupare uno storage slot mutabile.

Nel threat model: buyer, seller e price non possono essere cambiati da una futura funzione amministrativa.

L'immutabilità non rende corretta la scelta iniziale: se il constructor riceve un seller sbagliato, l'errore resta permanente. Per questo validiamo gli input del constructor.

3.5 · IL CONSTRUCTOR

La prima transizione: da «non esiste» a Created

```
buyer  = msg.sender del deployment  
seller = seller scelto  
price  = prezzo scelto  
state  = Created
```

INVARIANTI DA STABILIRE SUBITO

```
seller ≠ address(0)  
price > 0  
seller ≠ buyer  
state = Created
```

Un invariant che non è vero subito dopo la costruzione non potrà essere salvato dai test delle funzioni successive.

3.6 • MODIFICATORI

Utili, ma non magici

```
modifier onlyBuyer() {  
    if (msg.sender != buyer) revert Unauthorized(msg.sender);  
    -;  
}  
  
modifier onlyState(State expected) {  
    if (state != expected) revert WrongState(expected, state);  
    -;  
}
```

SI LEGGE COSÌ

1. verifica buyer
2. verifica stato
3. esegui il body

MA ATTENZIONE

- una funzione può dimenticare il modifier
- l'ordine dei modifier può essere significativo
- un modifier può nascondere logica ed effetti
- il nome può promettere più di ciò che verifica

Regola da auditor: non fidarti del nome onlyX. Apri sempre il modifier e leggi cosa controlla davvero.

3.7 · CUSTOM ERRORS

Dare un nome preciso alla proprietà violata

```
error Unauthorized(address caller);
error WrongState(State expected, State actual);
error WrongValue(uint256 expected, uint256 actual);

if (msg.sender ≠ buyer) {
    revert Unauthorized(msg.sender);
}
```

DUE VANTAGGI

Danno un nome preciso alla proprietà violata.
Portano dati utili senza lunghe stringhe di revert.

PROPRIETÀ VIOLATA



CUSTOM ERROR



TEST expectRevert

3.8 • PAYABLE E MSG.VALUE

La transizione non dipende solo da chi chiama

```
function fund() external payable ...  
  
if (msg.value ≠ price) {  
    revert WrongValue(price, msg.value);  
}
```

Stato + caller + valore	Esito
Created + buyer + price esatto	Funded
Created + buyer + 0 ETH	revert
Created + buyer + price - 1	revert
Created + buyer + price + 1	revert
Created + outsider + price	revert
Funded + buyer + price	revert

È già un piccolo **input space** multidimensionale: stato, identità e valore si combinano, e ogni combinazione ha un esito atteso.

3.9 · RICEVERE ETHER

Ricevere non è chiamare: il call graph di oggi

QUELLO CHE ACCADE

```
BUYER / test harness  
  |  
  | CALL con value  
  ↓  
ESCROW.fund()
```

QUELLO CHE NON ACCADE

```
Escrow → seller  
Escrow → token  
Escrow → indirizzo arbitrario
```

PERCHÉ CONTA OGGI

Nel body delle nostre funzioni l'Escrow non cede mai il controllo a codice esterno: possiamo studiare la state machine senza introdurre la reentrancy.

3.10 · STATI TERMINALI

Da Cancelled e ReleaseApproved non si esce

Cancelled

ReleaseApproved

Terminale significa: **non esiste alcuna transizione di uscita valida**. Una volta raggiunti, ogni funzione state-changing del nostro protocollo deve fallire.

GLI STATI TERMINALI SONO UN OTTIMO POSTO DOVE CERCARE BUG

- esiste una funzione dimenticata che può «riaprire» l'Escrow?
- si può finanziare dopo la cancellazione?
- si può approvare due volte?
- si può cambiare ruolo o prezzo dopo la chiusura?

3.11 · TRANSITION MATRIX

Una tabella stato × funzione, quasi una specifica eseguibile

Stato corrente	fund()	approveRelease()	cancelBeforeFunding()
Created	buyer + prezzo esatto: OK → Funded	REVERT	buyer: OK → Cancelled
Funded	REVERT	buyer: OK → ReleaseApproved	REVERT
ReleaseApproved	REVERT	REVERT	REVERT
Cancelled	REVERT	REVERT	REVERT

Ogni cella dovrebbe avere almeno un test, o essere coperta da una proprietà più generale. Le celle scoperte sono la prima cosa da cercare in un audit.

3.12 · THREAT MODEL DELLA V1

Cosa stiamo proteggendo, e cosa abbiamo escluso

ASSET

Ether custodito

dal contratto dopo fund()

Integrità dello stato

niente Funded senza la
sequenza corretta

Identità dei ruoli

restano quelli del deployment

Prezzo concordato

resta quello iniziale

ATTORI

```
buyer    → fund, approveRelease, cancelBeforeFunding  
seller   → passivo in questa versione  
outsider → nessuna funzione privilegiata
```

INPUT CONTROLLATI DAL CHIAMANTE

```
msg.sender · msg.value  
quale funzione · ordine delle chiamate
```

ASSUNZIONI DELIBERATE DELLA LEZIONE

solo Foundry/Anvil locale

nessun payout

nessun token

nessun arbitro

nessun timeout

nessuna upgradeability

nessun oracolo

non modelliamo Ether forzato dall'esterno

Stati, errori, eventi, configurazione

```
// pragma solidity 0.8.37;
enum State {
    Created, Funded, ReleaseApproved, Cancelled
}

error ZeroSeller();
error BuyerEqualsSeller();
error ZeroPrice();
error Unauthorized(address caller);
error WrongState(State expected, State actual);
error WrongValue(uint256 expected, uint256 actual);
```

```
event Funded(address indexed buyer, uint256 amount);
event ReleaseApproved(address indexed buyer);
event Cancelled(address indexed buyer);

address public immutable buyer;
address public immutable seller;
uint256 public immutable price;

State public state;
```

Tre configurazioni immutabili, un solo pezzo di stato mutabile: **state**. Tutta la sicurezza della lezione ruota intorno a chi lo cambia, e quando.

4 • IL CONSTRUCTOR

Validare prima di fissare per sempre

```
constructor(address seller_, uint256 price_) {  
    if (seller_ == address(0)) revert ZeroSeller();  
    if (seller_ == msg.sender) revert BuyerEqualsSeller();  
    if (price_ == 0) revert ZeroPrice();  
  
    buyer = msg.sender;  
    seller = seller_;  
    price = price_;  
    state = State.Created;  
}
```

DOMANDA DA AUDITOR

Perché **buyer** non è un parametro?

Perché la specifica di questa versione dice: il creatore del contratto è il buyer. È una scelta di protocol design, non una legge di Solidity.

Il constructor non è payable: creazione e funding restano due transizioni distinte.

4 · LE TRE FUNZIONI

Ogni funzione è una freccia del grafo

```
function fund() external payable onlyBuyer onlyState(State.Created) {  
    if (msg.value ≠ price) revert WrongValue(price, msg.value);  
    state = State.Funded;  
    emit Funded(msg.sender, msg.value);  
}  
  
function approveRelease() external onlyBuyer onlyState(State.Funded) {  
    state = State.ReleaseApproved;  
    emit ReleaseApproved(msg.sender);  
}  
  
function cancelBeforeFunding() external onlyBuyer onlyState(State.Created) {  
    state = State.Cancelled;  
    emit Cancelled(msg.sender);  
}
```

Created → Funded

Funded →

ReleaseApproved

Created → Cancelled

Le guardie sono i modifier: la freccia valida è scritta nella firma della funzione.

5.2 • ANALISI DI FUND()

Le domande da auditor, applicate

Domanda	Risposta
Chi può chiamarla?	Solo il buyer: il controllo effettivo è <code>msg.sender == buyer</code>
Input del chiamante	Nessun parametro ABI, ma <code>msg.sender</code> , <code>msg.value</code> e il momento della chiamata
Valore in ingresso	<code>msg.value</code> deve essere esattamente <code>price</code> : non <code>price - 1</code> , non <code>price + 1</code> , non 0
Stato letto	<code>buyer</code> , <code>state</code> , <code>price</code>
Stato modificato	<code>state = Funded</code> ; inoltre il balance ETH del contratto aumenta per effetto della call
Chiamate esterne	Nessuna: il body non presenta superficie di reentrancy da outbound call

PROPRIETÀ DERIVATA, DI TIPO STORICO

Se `state == Funded` attraverso le API previste, allora `fund()` è stata completata dal buyer con `msg.value == price`.

Due frecce, e i getter automatici

approveRelease()

Solo il buyer, nessun parametro, non payable. Legge buyer e state, scrive state = ReleaseApproved.

```
onlyState(State.Funded)
```

Non è un controllo difensivo accessorio: è parte **semantica** della funzione. Togliarlo cambia il protocollo.

cancelBeforeFunding()

Solo il buyer, nessun valore. L'unico input reale è il momento della chiamata. Scrive state = Cancelled.

```
Created → Cancelled    valido  
Funded  → Cancelled    non valido
```

Quando introdurremo i refund veri, questa parte del modello cambierà.

Getter automatici buyer(), seller(), price(), state(): leggono e basta. Ma la leggibilità on-chain non è segretezza: stato e bytecode sono comunque osservabili, anche senza un getter esplicito.

Le prime frasi verificabili

DOPO UNA COSTRUZIONE RIUSCITA

```
buyer = deployer  
seller ≠ address(0)  
seller ≠ buyer  
price > 0  
state = Created
```

E poiché sono immutable: buyer, seller e price non cambiano per tutta la vita del contratto.

ACCESS CONTROL

- un indirizzo diverso dal buyer non può finanziare
- non può approvare il release
- non può cancellare l'Escrow

Queste proprietà **non** dicono che il buyer possa sempre compiere quelle azioni. L'autorizzazione è necessaria, non sufficiente: serve anche lo stato corretto.

6.3 · PROPRIETÀ DELLA STATE MACHINE

Nove frasi che i test devono verificare

1

fund() può riuscire soltanto da Created

2

approveRelease() può riuscire soltanto da Funded

3

cancelBeforeFunding() può riuscire soltanto da Created

4

ReleaseApproved è raggiungibile soltanto passando da Funded

5

Cancelled è terminale

6

ReleaseApproved è terminale, in questa versione

7

l'Escrow non può essere finanziato due volte

8

un Escrow cancellato non può essere finanziato

9

un Escrow finanziato non può essere cancellato

6.4 • PROPRIETÀ SUL VALORE

Il prezzo esatto, e cosa resta dopo un revert

- un funding riuscito accetta esattamente price wei
- un funding con valore inferiore deve fallire
- un funding con valore superiore deve fallire
- se fund() reverte, la macchina non passa a Funded
- se fund() reverte, msg.value non resta accreditato al contratto come effetto parziale di quella call

L'ultima proprietà deriva dall'atomicità del revert vista nella Lezione 1: non basta che la call fallisca, deve non lasciare tracce.

Uno stato è una storia compressa

TENTAZIONE, DI NUOVO

```
address(escrow).balance = price
```

Utile come assert di scenario sui percorsi API normali. Ma il balance di un contratto può essere influenzato da meccanismi che non passano dalla sua normale funzione: non è un buon invariant universale.

STORIA COMPRESSA

```
state = ReleaseApproved
```

↓ implica

```
Created → Funded → ReleaseApproved
```

Se il codice permette Created → ReleaseApproved, lo stato non rappresenta più fedelmente la storia del protocollo.

Non confondere **assert di scenario** con **invariant universale**: la distinzione diventerà decisiva quando passeremo all'invariant testing.

7.1 · LABORATORIO FOUNDRY, TUTTO LOCALE

Due contratti, due suite di test

```
smart-contract-security-course/
├─ foundry.toml
├─ lib/
│   └─ forge-std/
├─ src/
│   ├── EscrowStateMachine.sol
│   └─ labs/
│       └─ BadEscrowStateMachine.sol
└─ test/
    ├── EscrowStateMachine.t.sol
    └─ BadEscrowStateMachine.t.sol
```

Oppure, per un progetto nuovo solo per il laboratorio:

```
forge init escrow-state-machine-lab
cd escrow-state-machine-lab
```

```
# foundry.toml
[profile.default]
src = "src"
out = "out"
libs = ["lib"]
solc_version = "0.8.37"
```

7.3 · LA SUITE

setUp: tre identità, fondi fittizi, un Escrow nuovo

```
uint256 internal constant PRICE = 5 ether;

function setUp() public {
    buyer = makeAddr("buyer");
    seller = makeAddr("seller");
    outsider = makeAddr("outsider");

    vm.deal(buyer, 100 ether);
    vm.deal(outsider, 100 ether);

    vm.prank(buyer);
    escrow = new EscrowStateMachine(seller, PRICE);
}
```

TESTINITIALCONFIGURATION

buyer, seller e price come attesi, e stato Created: il punto di partenza di ogni altro test.

VM.DEAL NON È UNA FEATURE DEL PROTOCOLLO

CHEATCODE = potere del tester
CONTRACT API = potere dell'utente

7.3 · IL PERCORSO VALIDO

Un test per ogni freccia del grafo

```
function testBuyerCanFundExactPrice() public {
    vm.prank(buyer);
    escrow.fund{value: PRICE}();

    assertEq(uint256(escrow.state()),
              uint256(EscrowStateMachine.State.Funded));
    assertEq(address(escrow).balance, PRICE);
}
```

TESTBUYERCANAPPROVEAFTERFUNDING

Dopo il funding, il buyer approva: stato ReleaseApproved e balance ancora PRICE.

TESTBUYERCANCANCELBEFOREFUNDING

Cancellazione da Created: stato Cancelled e balance a zero.

Tre frecce valide, tre test: la matrice inizia a riempirsi.

7.3 · LE FRECCHE PROIBITE

Il revert, e poi lo stato che non è cambiato

```
function testCannotApproveBeforeFunding() public {  
    vm.startPrank(buyer);  
  
    vm.expectRevert(EscrowStateMachine.WrongState.selector);  
    escrow.approveRelease();  
  
    vm.stopPrank();  
  
    assertEq(uint256(escrow.state()),  
            uint256(EscrowStateMachine.State.Created));  
}
```

GLI ALTRI TEST NEGATIVI

outsider non può finanziare · seller non può approvare · valore sbagliato · doppio funding · cancel dopo il funding.

TESTREVERTEDFUNDINGLEAVESNOPARTIALEFFECTS

Salva i saldi prima della call, poi verifica buyer.balance, balance dell'Escrow e stato: tutto invariato.

Nota tecnica: **expectRevert con il solo selector** richiede la corrispondenza esatta del revert data, quindi funziona con errori senza argomenti. Per WrongState, WrongValue e Unauthorized serve abi.encodeWithSelector con gli argomenti attesi, oppure vm.expectPartialRevert.

I comandi, e cosa cercare nella trace

<code>forge fmt</code>	formatta prima della review
<code>forge build</code>	deve compilare senza errori
<code>forge test</code>	tutta la suite
<code>forge test --match-contract EscrowStateMachineTest</code>	solo il contratto di oggi
<code>forge test --match-test testBuyerCanFundExactPrice -vvvv</code>	trace dettagliata del funding

```
buyer
| value: 5 ether
↓
Escrow::fund()
  +-- onlyBuyer
  +-- state = Created
  +-- msg.value = price
  +-- SSTORE state
  +-- LOG Funded
  ↓
return
```

Non dovresti vedere alcuna call dall'Escrow verso il seller: è il punto di confronto con la Lezione 4.

8 • TEST NEGATIVI

Quattro categorie riutilizzabili in ogni audit

A • CALLER SBAGLIATO

outsider → fund
seller → approveRelease
outsider → cancel

B • VALUE SBAGLIATO

PRICE - 1
PRICE + 1

C • STATO SBAGLIATO

Created → approve
Funded → fund
Funded → cancel
Cancelled → fund

D • RIPETIZIONE

fund due volte
approve due volte
cancel due volte

Valid paths:

Created → Funded → ReleaseApproved
Created → Cancelled

Everything else:

forbidden

Un auditor prova sistematicamente a trasformare il «forbidden» in «reachable».

Il bug: una guardia di stato che manca

```
function fund() external payable {
    if (msg.sender != buyer) revert Unauthorized(msg.sender);
    if (state != State.Created) revert WrongState();
    if (msg.value != price) revert WrongValue();
    state = State.Funded;
}

function approveRelease() external {
    if (msg.sender != buyer) revert Unauthorized(msg.sender);

    // BUG: manca state == State.Funded
    state = State.ReleaseApproved;
}
```

Il bug **non** è «chiunque può chiamare»: il controllo del buyer c'è.

È «il buyer può chiamare nel momento sbagliato».

Created --approveRelease() →
ReleaseApproved

freccia inattesa

9.3 · RIPRODUZIONE LOCALE

Un test che passa, e proprio per questo dimostra il bug

```
function testDemonstratesInvalidTransition() public {
    assertEq(uint256(badEscrow.state()),
              uint256(BadEscrowStateMachine.State.Created));
    assertEq(address(badEscrow).balance, 0);

    // Nessun funding avviene.
    vm.prank(buyer);
    badEscrow.approveRelease();

    assertEq(uint256(badEscrow.state()),
              uint256(BadEscrowStateMachine.State.ReleaseApproved));
    assertEq(address(badEscrow).balance, 0);
}
```

```
forge test \
  --match-contract \
  BadEscrowStateMachineTest \
  -vvvv
```

Non stiamo dicendo «la suite è verde, quindi è sicuro».

Stiamo dicendo: **abbiamo scritto un esperimento che dimostra che il comportamento proibito è raggiungibile.**

9.4 · 9.5 · LA CORREZIONE

La guardia traduce la freccia valida

LA CORREZIONE

```
function approveRelease()  
  external  
  onlyBuyer  
  onlyState(State.Funded)  
{  
  state = State.ReleaseApproved;  
  emit ReleaseApproved(msg.sender);  
}
```

Funded → ReleaseApproved

IL TEST DI REGRESSIONE, GIÀ NELLA SUITE

```
function testCannotApproveBeforeFunding() public {  
  vm.startPrank(buyer);  
  vm.expectRevert(...WrongState...);  
  escrow.approveRelease();  
  vm.stopPrank();  
  
  // stato ancora Created  
}
```

Non serve soltanto oggi: serve a impedire che una modifica futura reintroduca il bug.

9.6 • PRIMA E DOPO

Cosa cambia davvero con una riga di guardia

Aspetto	Vulnerabile	Corretto
Controlla il caller	sì	sì
Controlla state == Funded	no	sì
Created → ReleaseApproved	possibile	revert
Funded → ReleaseApproved	possibile	possibile
Lo stato rappresenta la storia	no	sì, nel modello della lezione

Un access control corretto non compensa una state machine errata.

Il modello, le sequenze, chi può agire

Modello

- Quali sono tutti gli stati, espliciti e impliciti?
- Esiste un diagramma delle transizioni?
- Quali stati sono terminali?
- Quali stati dovrebbero essere irraggiungibili direttamente?

Sequenze

- Posso saltare una fase?
- Posso ripetere una fase?
- Posso tornare indietro da uno stato terminale?
- Posso chiamare nello stesso ordine ma con attori diversi?
- Una funzione valida da sola diventa invalida dopo un'altra?

Access control

- La funzione controlla il caller corretto?
- Controlla msg.sender o qualcosa di meno appropriato?
- Un ruolo ha più potere del necessario?
- Manca un modifier su qualche entry point parallelo?

Ogni entry point, il valore, la suite

Entry point

- Chi può chiamarla?
- In quali stati può essere chiamata?
- Quali input decide il caller?
- Accetta msg.value?
- Quali storage variable legge?
- Quali ne modifica?
- Quale nuovo stato rende raggiungibile?
- Può essere chiamata due volte?

Value

- Il valore deve essere esatto, minimo o massimo?
- Che cosa succede a valore zero?
- Che cosa succede a valore eccessivo?
- Un revert lascia davvero tutto invariato?

Test

- Esiste un test per ogni freccia valida?
- Esistono test per le frecce proibite importanti?
- Esistono test sulle ripetizioni?
- Esistono test con caller non autorizzati?
- Il test verifica lo stato dopo il revert?

Prima il grafo e le proprietà, poi il codice

1 • Transition matrix completa

Per ogni cella: SUCCESS con stato finale, o REVERT con motivo. Poi trova le celle senza test.

2 • Test mancante sul caller

L'outsider non può chiamare cancelBeforeFunding: verifica il revert e che lo stato resti Created.

3 • Stato terminale Cancelled

Dopo Cancelled, approveRelease reverte. Spiega perché fallisce per lo stato, pur avendo il caller giusto.

4 • Testare l'errore completo

Verifica l'intero payload di WrongValue(expected, actual). Quale trade-off rispetto al solo selector?

5 • Seller attivo

Nuovo stato Accepted: chi cancella da lì? Il seller può revocare? Prima il grafo, poi il codice.

6 • Mutation testing manuale

Togli onlyState(State.Created) da fund: quali test falliscono? Quale regression test aggiungeresti?

7 • Proprietà storica

«Se lo stato è ReleaseApproved, allora in passato...»: completala e trasformala in test deterministici.

8 • Threat modeling

Aggiungi il ruolo arbiter senza scrivere codice: quando entra, quali stati tocca, quali abusi introduce.

Otto idee da portare via

1 · La sequenza è parte della sicurezza

Autorizzata per il caller giusto, eseguibile nello stato sbagliato.

2 · Disegna prima la state machine

Stati e transizioni nascono dalla specifica, non dal codice.

3 · Le frecce assenti contano

I test negativi provano le transizioni che non devono essere raggiungibili.

4 · Chi e quando sono dimensioni diverse

Access control e state guard devono essere entrambi corretti.

5 · Uno stato racconta una storia

ReleaseApproved senza funding corrompe il significato dello stato.

6 · Il revert si testa negli effetti

Non basta sapere che la call è fallita.

7 · Scenario non è invariant universale

Un balance osservato in un percorso normale non è una verità globale.

8 · Il contratto di oggi è incompleto

L'Ether resta custodito: state machine separata dalle external call.

Da dove viene ogni affermazione

Fonte	Per che cosa
Solidity 0.8.37 Release Announcement	baseline del compilatore e bugfix
Solidity · Structure of a Contract	state variables, events, errors, enum
Solidity · Contracts	state mutability, immutable, custom errors
Solidity · Expressions and Control Structures	revert, atomicità, eccezioni
Solidity · SMTChecker	state properties e invarianti su più transazioni
ethereum.org · Anatomy of Smart Contracts	storage persistente, msg.sender
Foundry · Configuration · Reference · Guides	foundry.toml, cheatcode, workflow di testing
OWASP SC02:2026 · SCSVS-AUTH-2 · SCSVS-GOV-3	business logic, autorizzazione, transaction flow